Web Browser based uBITX VFO / BFO Controller - User Guide

VU2SPF - SP Bhatnagar

v4_7 – 22.05.2026

What Is This?

This program turns an ESP32 and Si5351 synthesizer chip into a *complete frequency controller* for an **uBITX transceiver**. It creates a WiFi access point that lets you control your radio from any smartphone, tablet, or computer browser - no special apps needed!
This may also be used as VFO/BFO etc for most analog Ham rigs by using appropriate frequencies generated by Si5351.

Attention: This program and its details are open sourced and provided as it is for experimenter amateur radio homebrewer to enjoy the hobby. Anyone can Modify / Upgrade and give it back to the community. No specific support to be expected. Collaborative suggestions and corrections are welcome (vu2spf at gmail dot com). TRY AT YOUR OWN RISK.

Hardware Requirements

- **ESP32** development board (any variant)
- **Si5351** clock generator module (I2C interface)
- **Optional**: Analog signal for S-meter (0-1.2V input on GPIO32)

Pin Connections

Pin	Purpose	Name in this program
ESP32 GPIO21	Si5351 SDA	
ESP32 GPIO22	Si5351 SCL	
ESP32 GPIO32	S-meter input (optional)	
Si5351 CLK0	12 MHz (Audio modulator/demodulator)	BFO2 in this program
Si5351 CLK1	33/57 MHz (Second IF mixer)	BFO1 in this program
Si5351 CLK2	45-75 MHz (First IF mixer / VFO)	VFO in this program

Getting Started

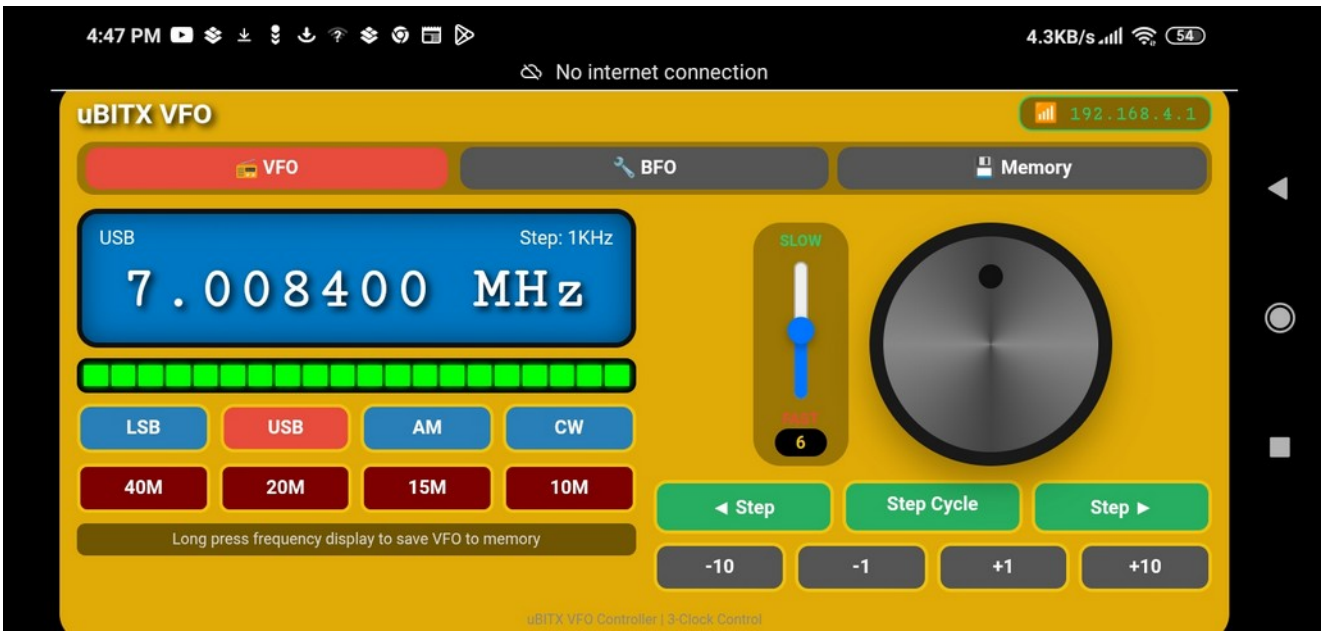
1. Upload the Code
- Install ESP32 board support in Arduino IDE
- Install required libraries: `Si5351`, `WiFi`, `WebServer`
- Compile and Upload to your ESP32

2. Connect to the ESP32 from a smartphone, computer, tablet etc by WiFi (there are other ways listed later)
- Power up the ESP32
- Look for WiFi network: **ubitx**
- Password: **12345678**
- Open browser and go to: **http://192.168.4.1**

There are 3 screens for VFO, BFOs and Memory, which can be selected from their corresponding tabs at the top.

3. Main Screen (VFO Tab)
To control the VFO (Clock2 of Si5351). The display shows the Operating frequency while the actual output frequency of Clock2 is as given in a previous section and explained in later section. Various contols on this screen are -

Control	Function
Knob	Drag/rotate to tune frequency
Vertical slider	Adjust knob sensitivity (SLOW → FAST)
LSB/USB/AM/CW	Select operating mode (changes BFO1 automatically) (AM/CW not implemented)
Step Cycle	Change tuning step (10Hz to 10MHz)
+/- 1/10 buttons	Quick frequency change (in 1x or 10x tuning steps)
Band buttons	Jump to ham bands (40m, 20m, 15m and 10M)
S-meter	Shows signal strength

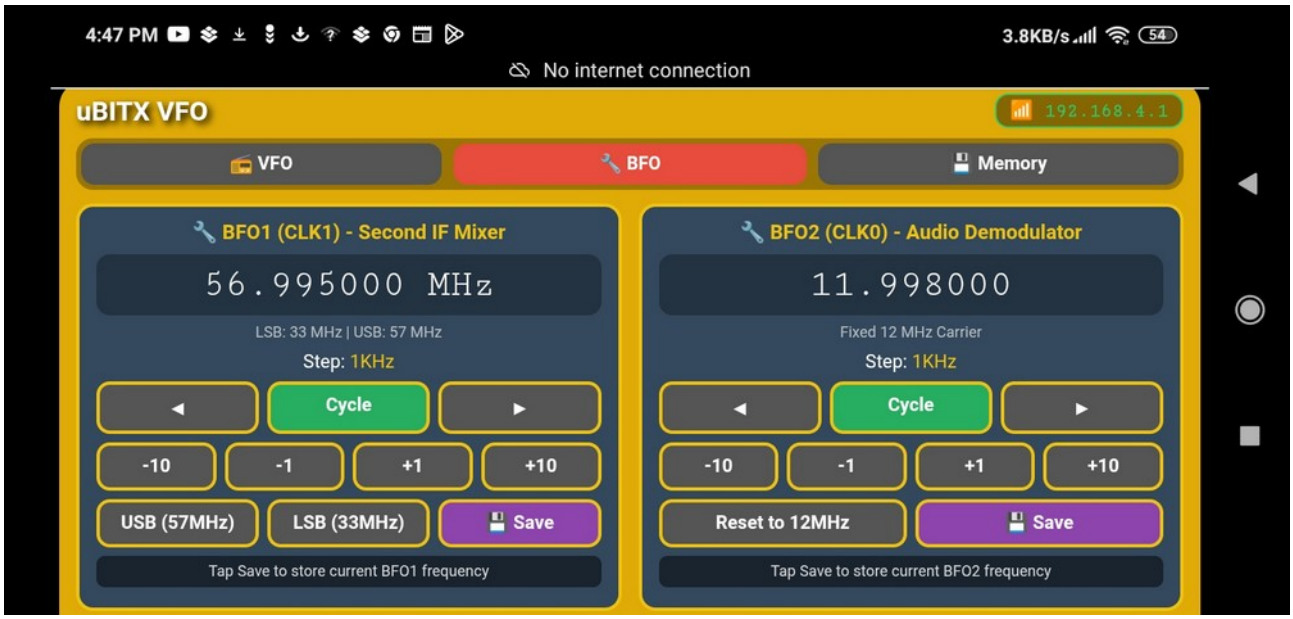


4. Saving VFO Frequency to Memory
To save current frequency:
- Long press (hold 0.5 seconds) on the frequency display
- Memory Tab (details below) is opened. Tap any M1-M10 button to save VFO frequencies in that Memory slot.

To Recall a saved frequency:
- Simply tap appropriate M1-M10 button on Memory Tab

Knob Sensitivity
- Move slider up (SLOW) → say when Fine tuning for CW/digital modes
- Move slider down (FAST) → like for Quick band scanning

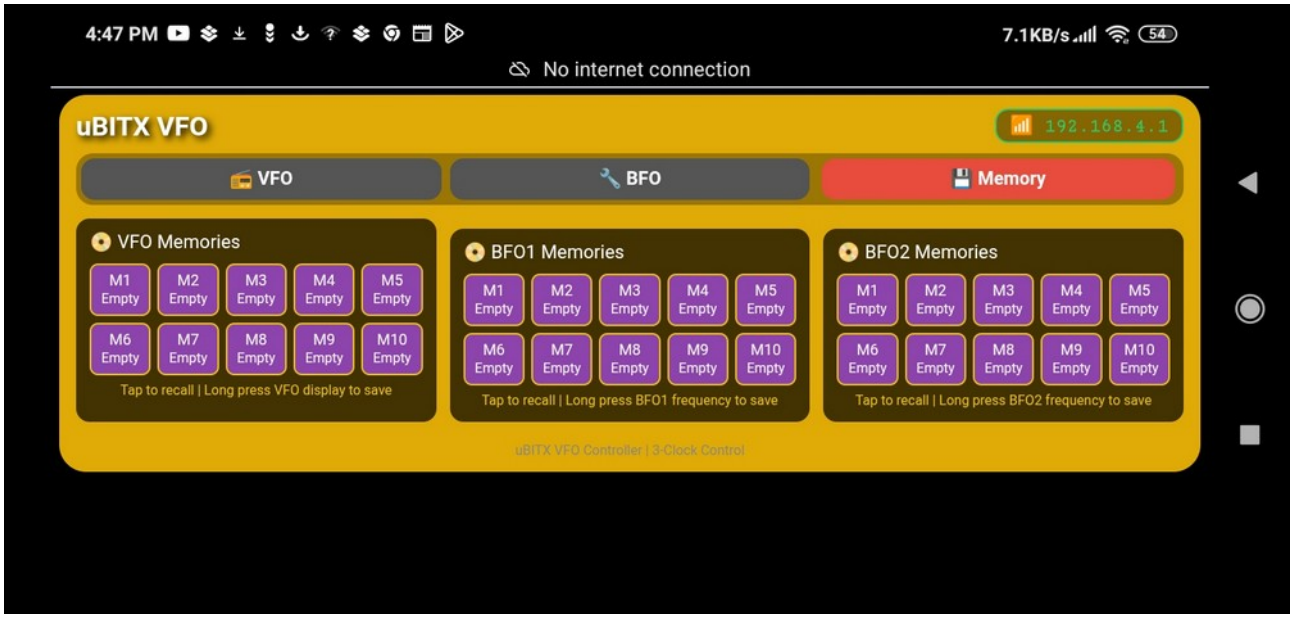
5. BFO Tab
Controls for the other two Si5351 Oscillators:
- BFO1 - Adjustable 1-60 MHz (USB 33MHz / LSB 57MHz presets- actual values as per raduino program)
- BFO2 - Adjustable 12 MHz carrier
- Independent step sizes for each BFO
- 10 memory slots per BFO



To adjust BFOs:
- Tap on BFO tab
- Use +/- buttons or Step Cycle button to select frequency step
- Tap Save Button
- In Memory Tab, Tap any memory button to save
- USB/LSB presets - Quick set for BFO1

To understand the required clock frequencies in ubitx look at a later section of this document.

6. Memory Tab
View and manage all 30 memory slots:
- VFO memories (10 slots) - Operating frequencies
- BFO1 memories (10 slots) - BFO1 frequencies
- BFO2 memories (10 slots) - BFO2 frequencies



Tap any memory button to recall that frequency.

Web Interface Features
- Mobile-optimized - Works on phones, tablets, desktops
- Touch-friendly - Large buttons, smooth knob control
- Real-time updates - No page refresh needed
- S-meter display - Shows signal strength (20-seg bar graph)

<u>Step Sizes Available for all Clocks</u>
- 10Hz, 100Hz, 1KHz, 5KHz, 10KHz, 100KHz, 1MHz, 10MHz

Si5351 Crystal Calibration (For Precision)
If you have a frequency counter, you can calibrate:
- Measure actual output at 10 MHz – on any Clock output of Si5351 (say BFO2-Clock0 after setting its frequency to10MHz)

- Calculate ppm error = ((measured - Expected) / Expected)* 100000
- correction = ppm_error × (-100)

- Modify the program within function “setup()”,

replace 0 with the calculated value of correction as below:
#define CRYSTAL_CORRECTION 0 // Replace with your measured value, 0 = no correction

In this program All Settings Are Saved
- VFO frequency
- Operating mode (USB/LSB/AM/CW)
- Step sizes for all three clocks
- BFO1 and BFO2 frequencies
- All 30 memory slots (10 per clock)
- Crystal calibration (if set)
Settings persist after power cycle!

Troubleshooting

Problem	Solution
Can't connect to WiFi	Check ESP32 power, wait 10 seconds, look for "ubitx" network
Wrong frequency on CLK0	Remove drive_strength settings for 12 MHz output (it was discovered by chance and is already disabled)
Knob not responding	Use touch or mouse drag on the knob circle
Memory not saving	Clear browser cache or try different browser
Frequencies not exact	Add crystal calibration: `si5351.set_correction(value, SI5351_PLL_INPUT_XO)

Credits
- The Original Smartphone VFO concept, idea and implementation by Mirko Pavleski (mircemk) for single clock control, it is published at
- https://www.hackster.io/mircemk/the-ultimate-smartphone-vfo-esp32-si5351-wireless-control-bc1fcd
- It was updated for uBITX adaptation with full 3-clock control and Web interface with smooth touch tuning, with the help of AI (Deepseek and Claude), by VU2SPF. Even Part of this document was generated by AI and later modified / edited by VU2SPF.
Enjoy your software-defined VFO! The entire radio frequency control is now at your fingertips from any device with a web browser.

73s
VU2SPF

How uBITX Architecture Works

This program implements the uBITX v3/4/5/6 frequency scheme:
(Carefully match frequencies with your hardware version – github.com/afarhan)

Si5351 Clock	Frequency (approx)	Purpose
CLK2	45-75 MHz	Main VFO - mixes with incoming signal to create 45 MHz IF
CLK1 [BFO1]	33 MHz (LSB) / 57 MHz (USB)	Second IF mixer - selects sideband
CLK0 [BFO2]	12 MHz	Audio demodulator - fixed carrier

For Rx -
1. The VFO (Clock2 of Si5351) is used to produce 45MHz First Intermediate Frequency (First IF) through first mixer of uBITX. (VFO > 45MHz so Sidebands Inverted).
2. After passing through a 45MHz filter the IF signal is mixed with BFO1(Clock 1)(LSB 33MHz < 45MHz so no Sideband Inversion, for USB 57MHz > 45MHz so Sideband Inversion) to generate 12MHz Second IF .
3. This 12 MHz IF is passed through xtal ladder to filter LSB and later demodulated by mixing with a near 12 MHz clock to generate audio.

For Tx-
Tx chain is exactly same as Rx but in reverse order.
1. Audio is mixed with 12MHz BFO2 (Clock 0) and passed through 12MHz xtal filter to remove LSB.
2. This 12MHz is up-converted at second mixer into 45 MHz IF using LSB/USB clock 1
3. The first mixer using VFO (Clock2) produces SSB at operating frequency.

<u>Frequency Calculation</u>
USB Mode: VFO = Operating + 45 MHz
LSB Mode: VFO = Operating - 45 MHz

Operating Frequency	VFO Output
7.0 MHz (40m)	52.0 MHz
14.0 MHz (20m)	59.0 MHz
21.0 MHz (15m)	66.0 MHz
28.0 MHz (10m)	73.0 MHz

Different ways to connect your smartphone to ESP32 based controller

1. *Using ESP32 as Access Point* – Phone connects to ESP32 without needing a router.
This is the default method. To connect
- Power up the ESP32 wait for a few seconds.
- Look for WiFi network: **ubitx**
- Password: **12345678**
- Open browser and go to: **http://192.168.4.1**

Advantages:
Works completely standalone — no router, no internet needed in the shack
You connect your phone directly to the ESP32, open 192.168.4.1, and control it immediately
No dependency on your home WiFi being available
Lower latency since there's no router hop in between
The transceiver stays portable — works anywhere, even in the field
Potential considerations:
Your phone loses internet access while connected to ubitx (since the ESP32 doesn't route to the internet). On most Android phones this is handled gracefully; on iPhones it may warn you about "no internet connection" but still works fine for the controller page.
Only one device can practically control it at a time, which is fine for a VFO controller.

2. *Using regular home WiFi* –
If you ever wanted to integrate it into your home shack network (so your phone keeps internet), you could switch the ESP32 to Station mode (STA) and connect it to your home router instead — but that would require setting your home WiFi credentials in the progam (Home_ssid and Home_password) and also connect a pin named wifi_switch (currently GPIO 27 but you can use any free pin and change in the program) to Ground.

Again there are two different methods for connection using Wifi -
a. *Using mDNS*, which did not work (for me) on android but works well on IOS/ Linux/ and perhaps on Windows. and
b. *Using a fixed IP address*

Both are controlled by a variable called “fixedIP” in the program.
- When you comment out this variable (two slashes // in front of this variable) then it works in mDNS mode and you can access your vfo controller by typing “http://ubitx.local” (android did not work)

- When the “fixedIP” is not commented out the vfo is at “http://192.168.1.200”

For a ham radio VFO controller, the AP mode approach is the right call — simple, self-contained, and reliable. 73!